

IO

读入、输出优化

在默认情况下，`std::cin/std::cout` 是极为迟缓的读入/输出方式，而 `scanf/printf` 比 `std::cin/std::cout` 快得多。

▼ 注意

`cin/cout` 与 `scanf/printf` 的实际速度差会随编译器和操作系统的不同发生一定的改变。如果想要进行详细对比，请以实际测试结果为准。

下文将详细介绍读入输出的优化方法。

关闭同步/解除绑定

```
std::ios::sync_with_stdio(false)
```

这个函数是一个「是否兼容 `stdio`」的开关，C++ 为了兼容 C，保证程序在使用了 `printf` 和 `std::cout` 的时候不发生混乱，将输出流绑定到了一起。同步的输出流是线程安全的。

这其实是 C++ 为了兼容而采取的保守措施，也是使 `cin/cout` 速度较慢的主要原因。我们可以在进行 IO 操作之前将 `stdio` 解除绑定，但是在这样做之后要注意不能同时使用 `std::cin` 和 `scanf`，也不能同时使用 `std::cout` 和 `printf`，但是可以同时使用 `std::cin` 和 `printf`，也可以同时使用 `scanf` 和 `std::cout`。

tie

`tie` 是将两个 `stream` 绑定的函数，空参数的话返回当前的输出流指针。

在默认的情况下 `std::cin.tie()` 绑定的是 `&std::cout`，每次进行格式化输入的时候都要调用 `std::cout.flush()` 清空输出缓冲区，这样会增加 IO 负担。可以通过 `std::cin.tie(nullptr)` 来解除绑定，进一步加快执行效率。

但需要注意的是，在解除了 `std::cin` 和 `std::cout` 的绑定后，程序中必须手动 `flush` 才能确保每次 `std::cout` 展现的内容可以在 `std::cin` 前出现。这是因为 `std::cout` 被 `buffer` 为默认设置。例如：

```
1 std::cout << "Please input your name: "
2         << std::flush; // 或者: std::endl;
3         // 因为每次调用 std::endl 都会 flush 输出缓冲区，而 \n
4         // 则不会。
5 // 但请谨慎使用，过多的 flush 会影响程序效率
6 std::cin >> name;
```

代码实现

```
1 std::ios::sync_with_stdio(false);
2 std::cin.tie(nullptr);
```

读入优化

`scanf` 和 `printf` 依然有优化的空间，这就是本章所介绍的内容——读入和输出优化。

- 注意，本页面中介绍的读入和输出优化均针对整型数据，若要支持其他类型的数据（如浮点数），可自行按照本页面介绍的优化原理来编写代码。

原理

众所周知，`getchar` 是用来读入 1 byte 的数据并将其转换为 `char` 类型的函数，且速度很快，故可以用「读入字符——转换为整型」来代替缓慢的读入。

每个整数由两部分组成——符号和数字。

整数的 '+' 通常是省略的，且不会对后面数字所代表的值产生影响，而 '-' 不可省略，因此要进行判定。

10 进制整数中是不含空格或除 0~9 和正负号外的其他字符的，因此在读入不应存在于整数中的字符（通常为空格）时，就可以判定已经读入结束。

C 和 C++ 语言分别在 `ctype.h` 和 `cctype` 头文件中，提供了函数 `isdigit`，这个函数会检查传入的参数是否为十进制数字字符，是则返回 **true**，否则返回 **false**。对应的，在下面的代码中，可以使用 `isdigit(ch)` 代替 `ch >= '0' && ch <= '9'`，也可以使用 `!isdigit(ch)` 代替 `ch < '0' || ch > '9'`。

代码实现

```
1 int read() {
2     int x = 0, w = 1;
3     char ch = 0;
4     while (ch < '0' || ch > '9') { // ch 不是数字时
5         if (ch == '-') w = -1;    // 判断是否为负
6         ch = getchar();          // 继续读入
7     }
8     while (ch >= '0' && ch <= '9') { // ch 是数字时
9         x = x * 10 + (ch - '0'); // 将新读入的数字「加」在 x 的后面
10        // x 是 int 类型，char 类型的 ch 和 '0' 会被自动转为其对应的
11        // ASCII 码，相当于将 ch 转化为对应数字
12        // 此处也可以使用 (x<&LT3)+(x<&LT1) 的写法来代替 x*10
13        ch = getchar(); // 继续读入
14    }
15    return x * w; // 数字 * 正负号 = 实际数值
16 }
```

- 举例

读入 `num` 可写为 `num=read();`。

输出优化

原理

同样是众所周知，`putchar` 是用来输出单个字符的函数。

因此将数字的每一位转化为字符输出以加速。

要注意的是，负号要单独判断输出，并且每次 % (mod) 取出的是数字末位，因此要倒序输出。

代码实现

```

1 void write(int x) {
2     if (x < 0) { // 判负 + 输出负号 + 变原数为正数
3         x = -x;
4         putchar('-');
5     }
6     if (x > 9) write(x / 10); // 递归，将除最后一位外的其他部分放到递归中输出
7     putchar(x % 10 + '0'); // 已经输出（递归）完 x 末位前的所有数字，输出末位
8 }

```

但是递归实现常数较大的，我们可以写一个栈来实现这个过程。

```

1 void write(int x) {
2     static int sta[35];
3     int top = 0;
4     do {
5         sta[top++] = x % 10, x /= 10;
6     } while (x);
7     while (top) putchar(sta[--top] + 48); // 48 是 '0'
8 }

```

• 举例

输出 num 可写为 `write(num);`。

更快的读入/输出优化

通过 `fread` 或者 `mmap` 可以实现更快的读入。

`fread` 能将需要的文件部分读入内存缓冲区。`mmap` 则会调度内核级函数，将文件一次性地映射到内存中，类似于可以指针引用的内存区域。所以在日常程序读写时，只需要重复读取部分文件可以使用 `fread`，因为如果用 `mmap` 反复读取一小块文件，做一次性内存映射并且内核处理 `page fault` 的花费会远比使用 `fread` 的内核级函数调度大。

同时 `fread` 和 `mmap` 由于是整段整段读取、写入，所以比 `getchar()`/`putchar()` 要快的多。并且 `mmap` 确保了进程间自动共享，存储区如果可以也会与内核缓存分享信息，确保了更少的拷贝操作。

`fread` 类似于参数为 `"%s"` 的 `scanf`，不过它更为快速，而且可以一次性读入若干个字符（包括空格换行等制表符），如果缓存区足够大，甚至可以一次性读入整个文件。

对于输出，我们还有对应的 `fwrite` 函数。

```

1 std::size_t fread(void* buffer, std::size_t size, std::size_t count,
2                  std::FILE* stream);
3 std::size_t fwrite(const void* buffer, std::size_t size, std::size_t count,
4                  std::FILE* stream);

```

使用示例：`fread(buf, 1, SIZE, stdin)`，表示从 `stdin` 文件流中读入 `SIZE` 个大小为 1 byte 的数据块到 `Buf` 中。

读入之后的使用就跟普通的读入优化相似了，只需要重定义一下 `getchar`。它原来是从文件中读入一个 `char`，现在变成从 `Buf` 中读入一个 `char`，也就是头指针向后移动一位。

```

1 char buf[1 << 20], *p1, *p2;
2 #define gc() \
3     (p1 == p2 && (p2 = (p1 = buf) + fread(buf, 1, 1 << 20, stdin), p1 == p2) \
4     ? EOF \
5     : *p1++)

```

`fwrite` 也是类似的，先放入一个 `OutBuf[MAXSIZE]` 中，最后通过 `fwrite` 一次性将 `OutBuf` 输出。

参考代码：

```
1 namespace IO {
2 constexpr int MAXSIZE = 1 << 20;
3 char buf[MAXSIZE], *p1, *p2;
4 #define gc() \
5     (p1 == p2 && (p2 = (p1 = buf) + fread(buf, 1, MAXSIZE, stdin), p1 == p2) \
6         ? EOF \
7         : *p1++)
8
9 int rd() {
10     int x = 0, f = 1;
11     char c = gc();
12     while (!isdigit(c)) {
13         if (c == '-') f = -1;
14         c = gc();
15     }
16     while (isdigit(c)) x = x * 10 + (c ^ 48), c = gc();
17     return x * f;
18 }
19
20 char pbuf[1 << 20], *pp = pbuf;
21
22 void push(const char &c) {
23     if (pp - pbuf == 1 << 20) fwrite(pbuf, 1, 1 << 20, stdout), pp = pbuf;
24     *pp++ = c;
25 }
26
27 void write(int x) {
28     static int sta[35];
29     int top = 0;
30     do {
31         sta[top++] = x % 10, x /= 10;
32     } while (x);
33     while (top) push(sta[--top] + '0');
34 }
35 } // namespace IO
```

`mmap` 是 linux 系统调用，可以将文件一次性地映射到内存中。在一些场景下有更优的速度。

注意 `mmap` 不能在 Windows 环境下使用（例如 CodeForces 的 tester），同时也不建议在正式赛场上使用，可以在卡常时使用。在使用前要引入 `fcntl.h`, `unistd.h`, `sys/stat.h` 与 `sys/mman.h`。

读入示例：首先要获取文件描述符 `fd`，然后通过 `fstat` 获取文件信息以得到文件大小，此后通过 `char *pc = (char *)mmap(NULL, state.st_size, PROT_READ, MAP_PRIVATE, fd, 0)`；将指针 `*pc` 指向我们的文件。可以直接用 `*pc++` 替代 `getchar()`。

当我们要提交不使用文件操作的题目时，可以将 `fd` 设为 0，表示从 `stdin` 读入。但是，对 `stdin` 使用 `mmap` 是极其危险的行为，同时不能在终端输入，我们不建议您这么做。

▼ 参考代码

```

1 #include <LBits/stdc++.h>
2 #include <LTfcntl.h>
3 #include <LTsys/mman.h>
4 #include <LTsys/stat.h>
5 #include <LTunistd.h>
6 char *pc;
7
8 int rd() {
9     int x = 0, f = 1;
10    char c = *pc++;
11    while (!isdigit(c)) {
12        if (c == '-') f = -1;
13        c = *pc++;
14    }
15    while (isdigit(c)) x = x * 10 + (c ^ 48), c = *pc++;
16    return x * f;
17 }
18
19 int main() {
20     int fd = open("*.in", O_RDONLY);
21     struct stat state;
22     fstat(fd, &state);
23     pc = (char *)mmap(NULL, state.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
24     close(fd);
25     printf("%d", rd());
26 }

```

输入输出的缓冲

`printf` 和 `scanf` 是有缓冲区的。这也就是为什么，如果输入函数紧跟在输出函数之后/输出函数紧跟在输入函数之后可能导致错误。

刷新输出缓冲区的条件

1. 程序结束；
2. 关闭文件；
3. `printf` 输出 `\r` 或者 `\n` 到终端的时候（注：如果是输出到文件，则不会刷新缓冲区）；
4. 手动 `fflush()`；
5. 缓冲区满自动刷新；
6. `cout` 输出 `endl`；
7. 手动 `cout.flush()`。

使输入输出优化更为通用

如果你的程序使用多个类型的变量，那么可能需要写多个输入输出优化的函数。下面给出的代码使用 [C++ 中的 `template`](#) 实现了对于所有整数类型的输入输出优化。

```

1 // 声明 template 类, 要求提供输入的类型T, 并以此类型定义内联函数 read()
2 template <typename T>
3 T read() {
4     T sum = 0, fl = 1; // 将 sum, fl 和 ch 以输入的类型定义
5     int ch = getchar();
6     for (; !isdigit(ch); ch = getchar())
7         if (ch == '-') fl = -1;
8     for (; isdigit(ch); ch = getchar()) sum = sum * 10 + ch - '0';
9     return sum * fl;
10 }

```

如果要分别输入 `int` 类型的变量 `a`, `long long` 类型的变量 `b` 和 `__int128` 类型的变量 `c`, 那么可以写成:

```
1 a = read<int>();
2 b = read<long long>();
3 c = read<__int128>();
```

示例代码

```
1 // #define DEBUG 1 // 调试开关
2 struct IO {
3     #define MAXSIZE (1 << 20)
4     #define isdigit(x) (x >= '0' && x <= '9')
5     char buf[MAXSIZE], *p1, *p2;
6     char pbuf[MAXSIZE], *pp;
7     #if DEBUG
8     #else
9     IO() : p1(buf), p2(buf), pp(pbuf) {}
10
11     ~IO() { fwrite(pbuf, 1, pp - pbuf, stdout); }
12 #endif
13     char gc() {
14     #if DEBUG // 调试, 可显示字符
15         return getchar();
16     #endif
17         if (p1 == p2) p2 = (p1 = buf) + fread(buf, 1, MAXSIZE, stdin);
18         return p1 == p2 ? ' ' : *p1++;
19     }
20
21     bool blank(char ch) {
22         return ch == ' ' || ch == '\n' || ch == '\r' || ch == '\t';
23     }
24
25     template <class T>
26     void read(T &x) {
27         double tmp = 1;
28         bool sign = false;
29         x = 0;
30         char ch = gc();
31         for (; !isdigit(ch); ch = gc())
32             if (ch == '-') sign = 1;
33         for (; isdigit(ch); ch = gc()) x = x * 10 + (ch - '0');
34         if (ch == '.')
35             for (ch = gc(); isdigit(ch); ch = gc())
36                 tmp /= 10.0, x += tmp * (ch - '0');
37         if (sign) x = -x;
38     }
39
40     void read(char *s) {
41         char ch = gc();
42         for (; blank(ch); ch = gc());
43         for (; !blank(ch); ch = gc()) *s++ = ch;
44         *s = 0;
45     }
46
47     void read(char &c) { for (c = gc(); blank(c); c = gc()); }
48
49     void push(const char &c) {
50     #if DEBUG // 调试, 可显示字符
51         putchar(c);
52     #else
53         if (pp - pbuf == MAXSIZE) fwrite(pbuf, 1, MAXSIZE, stdout), pp = pbuf;
54         *pp++ = c;
```

```

55 #endif
56 }
57
58 template <class T>
59 void write(T x) {
60     if (x < 0) x = -x, push('-'); // 负数输出
61     static T sta[35];
62     T top = 0;
63     do {
64         sta[top++] = x % 10, x /= 10;
65     } while (x);
66     while (top) push(sta[--top] + '0');
67 }
68
69 template <class T>
70 void write(T x, char lastChar) {
71     write(x), push(lastChar);
72 }
73 } io;

```

注意事项:

- 关闭调试开关时使用 `fread()`, `fwrite()`, 退出时自动析构执行 `fwrite()`。开启调试开关时使用 `getchar()`, `putchar()`, 便于调试。
- 若要进行文件读写, 请在所有读写进行之前加入 `freopen()`。

▼ 例题: [洛谷 P10815【模板】快速读入](#)

读入 n 个范围在 $[-10^{18}, 10^{18}]$ 的整数, 求和并输出。其中 n 的数据保证对于序列的任何前缀, 这个前缀的和在 $[-10^{18}, 10^{18}]$ 位有符号整形的存储范围内。

参考

[cin.tie 与 sync_with_stdio 加速输入输出 - 码农场](#)

[C++ 高速化 - Heavy Watal](#)

['Re: mmap/mlock performance versus read' - MARC](#)

本页面最近更新: 2025/1/2 23:58:12, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [Enter-tainer](#), [H-J-Granger](#), [lr1d](#), [Marcythm](#), [NachtgeistW](#), [ouuan](#), [Tiger3018](#), [Xeonacid](#), [Anguei](#), [Cassius0924](#), [Catreap](#), [CCXXxi](#), [Chaigidel](#), [chaiHdu](#), [Chrogeek](#), [countercurrent-time](#), [Dev-XYS](#), [Gesruea](#), [HeRaNO](#), [i-yyi](#), [isdanni](#), [ksyx](#), [lcfsh](#), [Link-cute](#), [mgt](#), [scp020](#), [SkqLiao](#), [SukkaW](#), [Tiphereth-A](#), [voidge](#), [voidge](#), [XuYueming520](#), [YZircon](#), [ZnPdCo](#), [zryi2003](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供, 附加条款亦可能应用